

El semantic layer decide si tus data agents dicen la verdad

La generative BI confiable se gana en el contexto, no en el modelo

Ricardo Benavides · julio 2026 · deep-dive

Hay dos formas de fallar, y una es mucho peor

Un director pregunta: "¿cuánto le vendimos a clientes nuevos el trimestre pasado?". El asistente puede fallar de dos maneras. Puede devolver un error. O puede devolver un número que se ve impecable y está equivocado.

Lo primero es una molestia. Lo segundo termina en una lámina de comité y mueve una decisión en la dirección equivocada. Y así es precisamente como falla el text-to-SQL cuando lo dejas operar sin contexto sobre un data warehouse real.

Ese es el punto que se pierde en la conversación sobre generative BI: el problema casi nunca es el modelo de lenguaje. El modelo mejora cada trimestre y se abarata por sí solo. Lo que decide si puedes confiar en la respuesta es el contexto —qué significa cada métrica, cuál es la tabla correcta, qué business rule aplica—, y ese contexto no lo aporta el modelo. Lo aportas tú, en el semantic layer.

Lo que dicen los números

El benchmark 2026 de dbt Labs es la medición más clara que he visto sobre esto, y deja dos conclusiones sobre la mesa.

La primera: el text-to-SQL mejoró de forma notable. Sobre el set completo de preguntas pasó de 32.7% a 64.5% de acierto, casi el doble en una sola generación de modelos. Quien haya dicho "esto no funciona" hace dos años debería revisar su postura.

La segunda cambia el marco por completo. Para las preguntas que caen dentro del alcance de un semantic layer bien modelado, el acierto se acerca o llega al 100%. No porque el modelo sea más inteligente, sino porque la query se vuelve determinista: si las métricas y las dimensiones ya están definidas, el modelo no tiene por dónde introducir un resultado sutilmente equivocado.

Conviene tener presente el punto de partida. Sin ninguna capa de contexto, varias mediciones citadas ubican el acierto base del text-to-SQL en torno al 21-25%. Desde ahí, 64.5% con buen prompting es un salto enorme; ~100% dentro de un modelo semántico es otra categoría.

Aun así, el porcentaje no es lo más importante. Lo más importante es cómo falla cada uno. El semantic layer, cuando no puede responder, te lo dice. El text-to-SQL entrega una respuesta que suena bien y está equivocada, sin ninguna señal de alerta. Para una exploración puntual, esa flexibilidad se agradece. Pero para un número que va a un auditor, a un OKR o a un KPI de la compañía, la diferencia entre "eso no lo puedo responder" y un dato inventado con apariencia de verdad lo es todo.

Qué hace realmente un semantic layer

Bajemos de la abstracción, porque "semantic layer" suena a término técnico y en el fondo es una idea muy concreta.

Es una capa de traducción: toma un schema de producción críptico y sin documentar, y lo convierte en una interfaz de negocio, limpia y documentada, sobre la que la IA sí puede razonar con confianza. En lugar de exponer *raw transactions con veinte columnas de nombre indescifrable*, *expones una view de ingresos netos* que declara qué mide y qué reglas aplica.

La parte que casi todos subestiman: la lógica de negocio vive en la definición de la view, no en la memoria del modelo. Si la view de ingresos netos ya incluye `WHERE refundflag = 0 AND testaccount = 0`, cada query que pasa por ahí hereda esa regla automáticamente. El agente no tiene que conocerla, ni recordarla, ni inferirla. La impone el motor SQL, no la buena voluntad del modelo.

Dremio lo plantea sin rodeos: el semantic layer resuelve la clase de errores más común y de mayor impacto —tabla equivocada, columna equivocada, business rule ausente—, que afecta al 80% de las queries de negocio y es la más propensa a producir números que terminan en decisiones reales. No resuelve todo. Resuelve el 80% que más duele.

Por eso la industria dejó de hablar de modelos y empezó a hablar de contexto

No es casualidad que el anuncio más comentado de Databricks esta temporada no sea un modelo, sino Genie Ontology: una context layer viva que codifica cómo el negocio define sus propias métricas y sus propios términos, no solo los datos que conecta. Genie dejó de ser un asistente de chat para apoyarse en una ontología del negocio e integrarse donde la gente ya trabaja —Teams, Microsoft 365 Copilot, Excel—.

El activo que Databricks pone al centro no es la capacidad de generar SQL. Es el contexto gobernado que hace que ese SQL sea confiable. Un data product sin ese contexto es un GPS sin destino: te lleva muy rápido a cualquier parte. El modelo es un commodity; el semantic layer es la ventaja defendible.

Lo que esto significa si tu mundo es SAP

Aquí la conversación se vuelve personal para quienes llevamos años trabajando con SAP y ahora buscamos integrar información a un lakehouse.

Cuando integramos SAP con Databricks, la parte que más valor generó no fue el pipeline. Mover datos es un problema resuelto. El valor estuvo en el trabajo lento y poco vistoso de definir qué significa cada métrica —qué es exactamente un "cliente nuevo", qué entra y qué no en "venta neta"— y dejar esa definición gobernada en un solo lugar.

En su momento parecía deuda que pagábamos únicamente para que los reportes fueran consistentes. Visto desde 2026, era otra cosa: sin saberlo, estábamos construyendo el semantic layer que hoy vuelve confiable a cualquier agente que consultemos encima.

Cómo empezar sin intentar abarcarlo todo

La tentación, cuando queda claro que el semantic layer es el activo, es modelar todo de una vez. Es la forma más segura de no terminar nunca.

El camino que recomienda Dremio, y que además es el que funciona en la práctica, es empezar de forma acotada. Toma las cinco métricas que tu negocio pregunta con más frecuencia. Construye views documentadas y probadas para cada una. Corre el agente contra esas cinco y valida las salidas contra valores históricos que ya conoces. Cuando esas cinco funcionen de forma confiable, agregas otras cinco. El semantic layer crece por incrementos, y con él crece el terreno donde puedes confiarle trabajo a un agente.

No es glamoroso. Es justo el tipo de trabajo que la moda de la IA nos tentó a saltarnos, y justo el que decide si tu asistente de datos es una herramienta confiable o un generador muy elocuente de números equivocados.

El cierre

Si vas a invertir en generative BI este año, la pregunta no es qué modelo eliges. Los buenos ya están disponibles, se parecen entre sí y cada vez cuestan menos. La pregunta es cuánto de tu negocio está codificado en un semantic layer gobernado, porque ese porcentaje —y no el modelo— es el techo de lo que tus agentes pueden responder sin mentirte.

Empieza por cinco métricas. Modélaslas bien. Valida contra la historia. El resto es paciencia.

Fuentes

- dbt Labs — *Semantic Layer vs. Text-to-SQL: 2026 Benchmark Update*. <https://docs.getdbt.com/blog/semantic-layer-vs-text-to-sql-2026>
- Dremio (Alex Merced) — *Semantic Layer for AI Agents: Stop Getting the Numbers Wrong*. <https://www.dremio.com/blog/semantic-layer-for-ai-agents-stop-getting-the-numbers-wrong/>
- Databricks — *Introducing Genie One, Genie Ontology, and Genie Agents*. <https://www.databricks.com/blog/introducing-genie-one-genie-ontology-and-genie-agents>
- Análisis sobre Snowflake Cortex Sense (línea base de text-to-SQL sin contexto, ~21-25%). <https://dev.to/albertomontagnese/text-to-sql-is-still-brittle-snowflakes-cortex-sense-is-a-new-take-2ahj>